

**ADVENTIST UNIVERSITY OF CENTRAL AFRICA**  
Faculty of Computing & Technology  
**In-Class Practical Quiz — Web Application Development**

---

Duration: 3 Hours    |    Stack: Spring Boot + React JS    |    Total Marks: 100  
Name: \_\_\_\_\_ ID: \_\_\_\_\_  
Date: \_\_\_\_\_

---

You will build a Library Book Borrowing System — a full-stack web application where members can borrow and return books in a campus library. The backend is Spring Boot and the frontend is React JS.

**Part 1 — Backend: Spring Boot REST API (40 pts)**

**Entities**

Create two entities: Book (id, title, author, category, available) and Borrow (id, memberName, memberId, bookId, borrowDate, returnDate).

**Setup (5 pts)**

- Spring Boot project with: Spring Web, Spring Data JPA, Postgresql
- Configure application.properties (Postgresql database, datasource, JPA settings)

**Book Endpoints (15 pts)**

| Method | Endpoint        | Description         |
|--------|-----------------|---------------------|
| GET    | /api/books      | Get all books       |
| GET    | /api/books/{id} | Get a single book   |
| POST   | /api/books      | Add a new book      |
| PUT    | /api/books/{id} | Update book details |

**Borrow Endpoints (15 pts)**

| Method | Endpoint          | Description                          |
|--------|-------------------|--------------------------------------|
| GET    | /api/borrows      | Get all borrow records               |
| POST   | /api/borrows      | Create a borrow record               |
| DELETE | /api/borrows/{id} | Delete a borrow record (return book) |

## **Part 2 — Frontend: React JS (40 pts)**

Build a React app that consumes the API above. Use functional components with `useState` and `useEffect` only. Basic CSS/inline styles are fine — focus on functionality.

### **Books Page (15 pts)**

- Fetch and display all books (title, author, category, availability)
- Include a form to add a new book — calls `POST /api/books` and refreshes the list

### **Borrow Page (15 pts)**

- Form fields: Member Name, Member ID, Book (dropdown from API), Borrow Date, Return Date (date inputs)
- On submit, call `POST /api/borrows` and show a success or error message

### **Borrow Records List (10 pts)**

- Display all borrow records in a table with a Return button per row
- Return calls `DELETE /api/borrows/{id}` and removes the row from the list

## **Part 3 — Integration & Code Quality (20 pts)**

| Criteria                                                    | Pts |
|-------------------------------------------------------------|-----|
| Frontend communicates with backend (CORS configured)        | 8   |
| API returns correct HTTP status codes (200, 201, 204, 404)  | 4   |
| No console errors; app does not crash on normal use         | 4   |
| Readable code: meaningful names, no dead/commented-out code | 4   |
|                                                             |     |

### **Deliverables**

- A zipped folder with both the Spring Boot project and the React project
- A `README.txt` with instructions on how to run both projects

### **Rules**

- Internet allowed ONLY for official docs: Spring, React, MDN
- AI tools are strictly prohibited (ChatGPT, Claude, Copilot, Gemini, etc.)
- No communication between students during the quiz
- Invigilators may ask you to explain any part of your code verbally

### **Suggested Time Breakdown**

| Phase | Task                                                | Time   |
|-------|-----------------------------------------------------|--------|
| 1     | Spring Boot: setup, entities, repositories, seeding | 40 min |
| 2     | Spring Boot: controllers, test with browser/Postman | 40 min |
| 3     | React: setup + Books page                           | 35 min |
| 4     | React: Borrow form + Borrow records list            | 35 min |
| 5     | CORS, final testing, README, submission             | 30 min |

### Grading Summary

| Part | Description                    | Points |
|------|--------------------------------|--------|
| 1    | Backend – Spring Boot REST API | 40     |
| 2    | Frontend – React JS            | 40     |
| 3    | Integration & Code Quality     | 20     |
|      | TOTAL                          | 100    |

*Focus on understanding, not memorizing. A simple working solution always beats a complex broken one. Good luck!*